

Data Visualization Module 2:

Introduction to D3 - Fundamental Technology: The Web – HTML – DOM – CSS – JavaScript
SVG. D3 Setup – Generating Page Elements – Binding Data - Drawing with data – Scales:
Domains and Ranges – Normalization – Creating a Scale – Scaling the Scatter Plot – Other
Methods and Other Scales. Axes – Modernizing the Chart – Update the Data – Transition
Updates – Interactivity.

Introduction to D3

D3.js stands for **Data-Driven Documents**.

It is a **JavaScript library** used to create **interactive, dynamic, and data-based visualizations** (charts, graphs, maps, dashboards) **directly in the browser**.

The **Web** is the platform where D3 runs.

HTML defines the **structure** of a webpage.

The **DOM** is a **tree-like representation** of HTML elements that JavaScript can access and modify.

CSS controls the **appearance** of elements.

JavaScript is the **engine** behind D3.

JavaScript provides:

- Variables
- Functions
- Loops
- Events
- Asynchronous loading (CSV, JSON)

SVG is an **XML-based graphics format** used to draw:

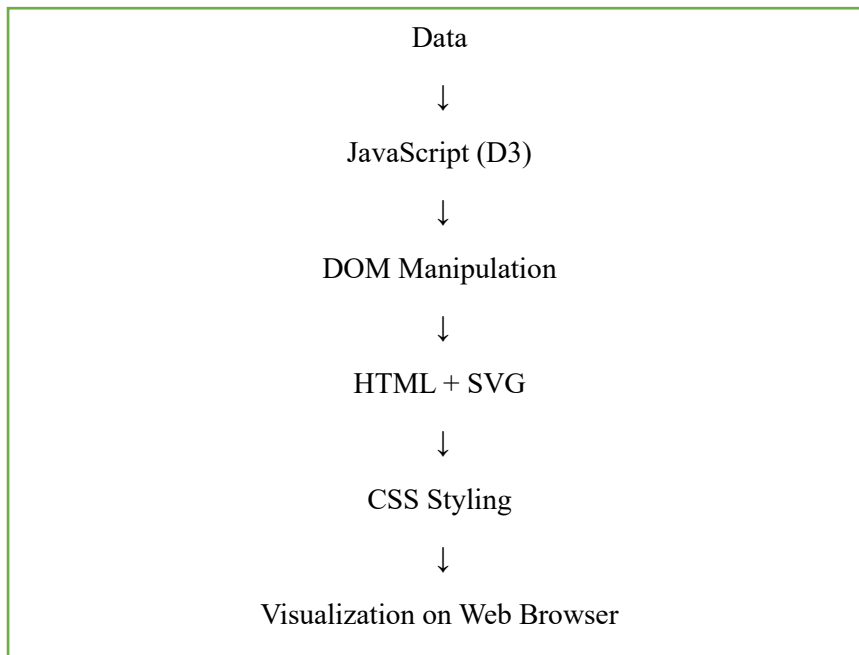
- Circles
- Rectangles
- Lines
- Paths
- Text

SVG is an **XML-based graphics format** used to draw:

- Circles

- Rectangles
- Lines
- Paths
- Text

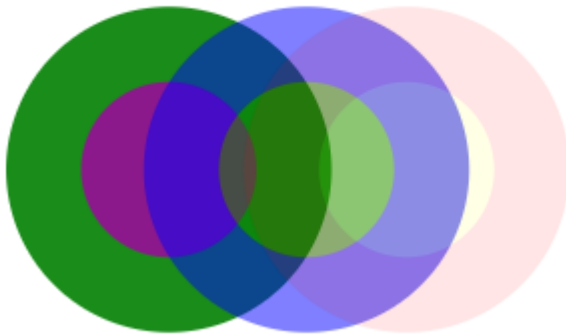
D3.js is a JavaScript library that binds data to the DOM and uses web standards like HTML, SVG, and CSS to create dynamic, interactive visualizations in the browser.



Program 1:

```
<svg width="500" height="500">
  <circle cx="125" cy="150" r="50" fill="purple" stroke="green"
stroke-width="30" opacity="0.9"/>
  <circle cx="180" cy="150" r="50" fill="green" stroke="blue"
stroke-width="30" opacity="0.5"/>
  <circle cx="220" cy="150" r="50" fill="yellow" stroke="red"
stroke-width="30" opacity="0.1"/>
</svg>
```

Output:



- ◆ `<svg width="500" height="500">`
 - Creates a **drawing canvas**
 - Width = 500px, Height = 500px
 - Origin (0,0) is at the **top-left corner**

Attribute	Meaning
cx	X-coordinate of center
cy	Y-coordinate of center
r	Radius
fill	Inner color
stroke	Border color
stroke-width	Border thickness
opacity	Transparency (0 → invisible, 1 → solid)

Program 2:

```

<html>
  <head>
    <title>D3 Rectangle</title>
    <script type="text/javascript" src="d3.v7.min.js"></script>

  </head>
  <body>

```

```
<div style="height: 200px;"></div>

<script>
  var w=500;
  var h=500;
  var dataset=[10,35,14,20,26];
  var svg=d3.select("body")
    .append("svg")
    .attr("width",w)
    .attr("height",h);

  svg.selectAll("rect")
    .data(dataset)
    .enter()
    .append("rect")
    .attr("x",function(d,i){
      return i*(w/dataset.length);
    })
    .attr("y",function(d){
      return h - d*5;
    })
    .attr("width",w/dataset.length-1)
    .attr("height",function(d){
      return d*5;
    })
    .attr("fill","teal");

  svg.selectAll("text")
    .data(dataset)
    .enter()
    .append("text")
    .attr("x",function(d,i){
      return i*(w/dataset.length);
```

```

    })

    .attr("y", function(d) {
        return h - d*5;
    })

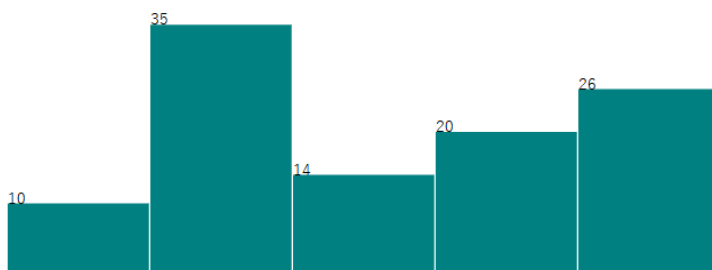
    .text(function (d)
    {
        return d;
    })

    .attr("text-anchor", "right")
    .attr("font-family", "sans-serif")
    .attr("font-size", "11px");

</script>
</body>
</html>

```

Output:



This program uses **D3.js** to create a simple **bar chart** inside an SVG element based on a given dataset. After loading the D3 library, the script defines the width and height of the SVG canvas as 500 pixels each and stores numerical values in an array called **dataset**. An SVG element is then dynamically appended to the HTML body using D3 and sized according to the defined dimensions. The program binds the dataset to rectangle (**rect**) elements using D3's data-driven approach, where one rectangle is created for each data value. The horizontal position of each bar is calculated using the index of the data item to ensure even spacing, while the vertical position is set as $h - d*5$ so that bars grow upward from the bottom of the SVG, accounting for SVG's inverted y-axis. The height of each bar is proportional to the data value, scaled by a factor of 5, and all bars are filled with a teal color. In addition to the bars, the program binds the same dataset to text elements to display the numerical values on the chart. Each text label is positioned near the top of its corresponding bar and styled using basic

font properties. Overall, the program demonstrates how D3 converts raw data into visual elements by binding data to SVG shapes and dynamically controlling their position and size.